

Draftly Reverse Engineering: Complete PRD, TRD & Implementation Guide

Executive Summary

This document provides a comprehensive reverse engineering analysis of Draftly, an AI-powered 3D scroll website builder, alongside a detailed Product Requirements Document (PRD) and Technical Requirements Document (TRD) for building a similar platform. The analysis includes a gap assessment comparing your current project, Signhify, with Draftly's capabilities, plus an actionable implementation roadmap and cinematic 3D UI design framework.^{[1][2]}

Key Findings:

- Draftly uses AI video generation (Runway/Kling) with frame extraction (ffmpeg) to create scroll-synced animations without WebGL^{[3][4][5]}
- Core architecture: AI API → Video Generation → Frame Extraction → Canvas Rendering → Code Export^[6]
- Your Signhify project uses Three.js/React Three Fiber for 3D, which differs fundamentally from Draftly's frame-based approach
- Implementation gap: ~4-6 months for MVP with proper AI video integration and code generation capabilities

Part 1: Ethical Reconnaissance Checklist

Allowed Analysis Methods

✓ Permitted Activities:

1. **Public Interface Analysis:** Study the live website's features, UX flows, and visual design^[1]
2. **Network Traffic Inspection:** Analyze browser DevTools for API endpoints, request/response patterns
3. **Technology Detection:** Use tools like Wappalyzer, BuiltWith to identify tech stack
4. **Feature Documentation:** Screenshot and document user-facing functionality
5. **Pattern Recognition:** Study scroll animation techniques, frame rendering methods^[5]
6. **Architecture Inference:** Deduce system design from observable behaviors
7. **Competitive Research:** Analyze similar products (Framer, Webflow, v0.dev) for industry patterns

✗ Prohibited Activities:

1. Reverse engineering compiled code or obfuscated JavaScript

2. Scraping or downloading proprietary assets without permission
3. Attempting to access private APIs, databases, or backend systems
4. Reproducing copyrighted content, code, or design elements exactly
5. Circumventing authentication or access controls
6. Decompiling or extracting proprietary AI models

Research Methodology

Phase 1: Public Analysis (1-2 days)

- Document all visible features and user workflows
- Capture UI/UX patterns, design system elements
- Test free tier limitations and paid feature gates
- Analyze exported code structure and quality

Phase 2: Technical Investigation (2-3 days)

- Study frame extraction technique from AI videos^{[4][7]}
- Research AI video generation APIs (Runway, Kling, Sora)^{[8][9]}
- Analyze scroll animation implementation patterns^[^5]
- Investigate ffmpeg frame extraction optimization^[^10]

Phase 3: Architecture Mapping (3-4 days)

- Infer database schema from user data patterns
- Map API endpoints from network requests
- Document state management and data flow
- Identify third-party integrations

Part 2: Draftly Reverse Engineering Analysis

Core Features Breakdown

1. AI-Powered Website Generation^{[6][1]}

- **Input:** Natural language prompt describing desired website
- **Processing:** AI interprets requirements, generates HTML/CSS/JavaScript
- **Output:** Production-ready code with cinematic scroll effects
- **Technology:** Likely uses Claude/GPT-4 for code generation with custom system prompts

2. Preset Template Gallery^[^1]

- 7+ scroll-reactive templates across industries (SaaS, E-commerce, AI, Travel)
- Each preset includes:

- Pre-configured 3D scroll animations
- Brand-specific design systems (typography, colors, spacing)
- Ready-to-edit sections (hero, features, CTA, footer)
- Examples: Meridian (cinematic scroll), TripVault (travel app), OrbitCRM (agency)

3. AI Video to Scroll Animation Pipeline^{[7][3][^4]}

- **Step 1:** User provides text prompt → AI generates keyframe images
- **Step 2:** AI video model (Kling/Runway) creates 8-second transition video
- **Step 3:** FFmpeg extracts 400 frames as WebP images^{[10][5]}
- **Step 4:** Canvas-based scroll interpolation renders frames based on scroll position
- **Advantage:** No WebGL overhead, works on all devices, buttery smooth at 60fps

4. Iterative Chat Editor^{[6][1]}

- Real-time content editing through conversational interface
- Supports: color changes, section reordering, copy updates, media swaps
- Maintains context across conversation for complex edits
- Similar to v0.dev/Lovable's chat-based editing pattern

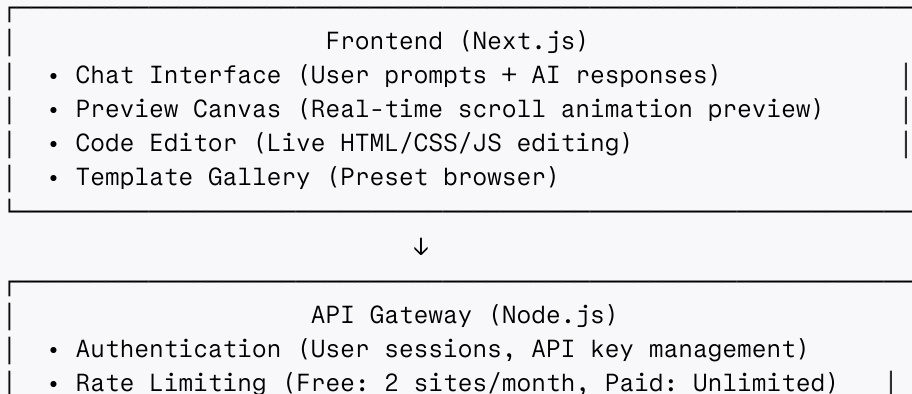
5. Multi-Video Continuation^[^1]

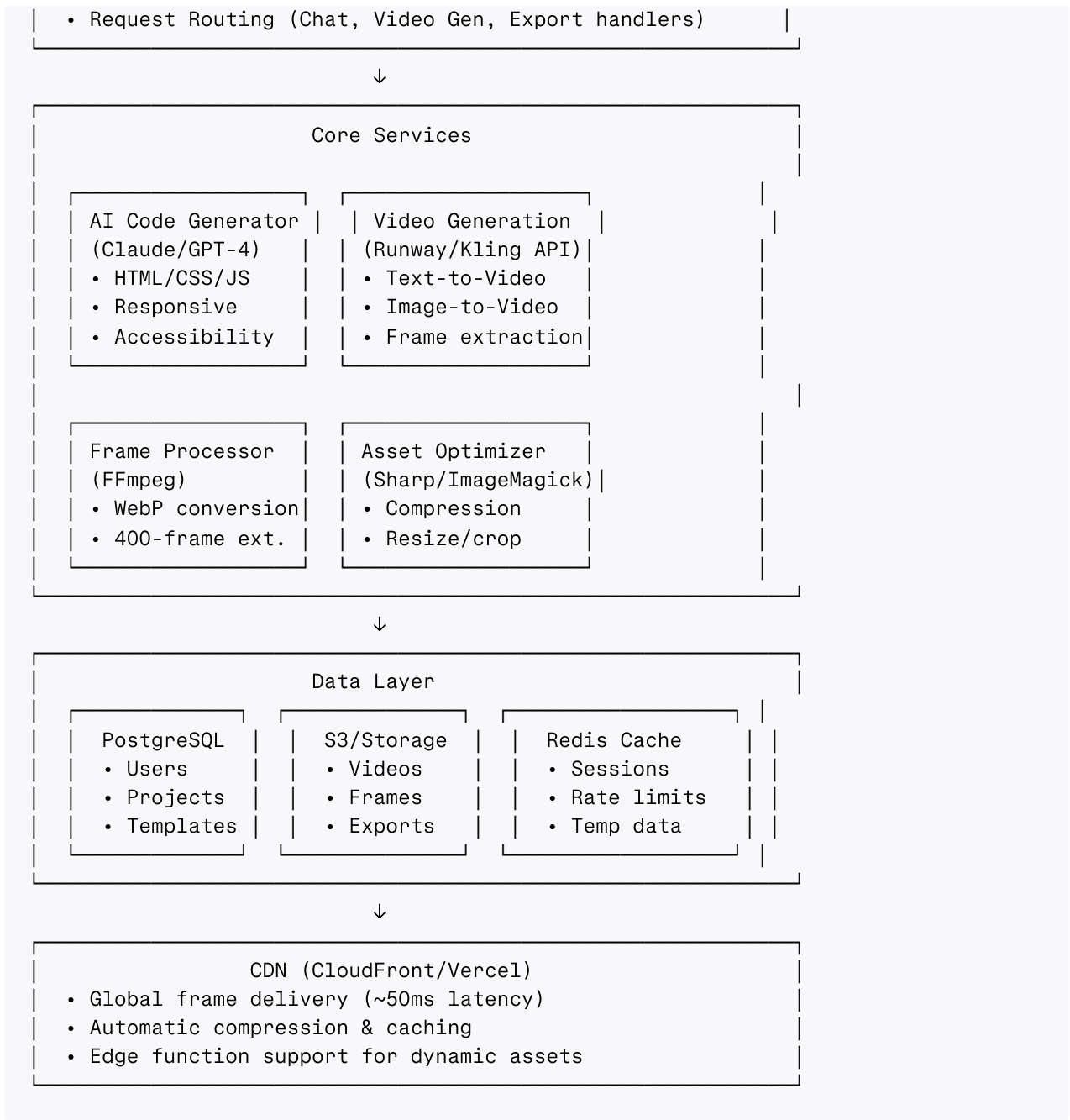
- Chain multiple 8-second videos for longer scroll experiences
- Automatic frame blending at video boundaries
- Supports up to 40+ seconds of scroll animation

6. Production Export^{[6][1]}

- **Frontend:** HTML/CSS/JS in optimized ZIP file
- **Full-Stack:** Optional Express.js backend starter
- **Performance:** Optimized frame loading, lazy loading, preload strategies
- **Adjustable FPS:** 10-40 fps slider for performance vs. smoothness trade-off

Technical Architecture (Inferred)





Frame-Based Scroll Animation Deep Dive

How It Works:[^5]

1. **Frame Extraction:** FFmpeg converts 8s video → 400 WebP images at 50fps
2. **Preloading:** All frames load in batches (avoid browser connection limits)
3. **Canvas Rendering:** JavaScript Canvas API draws frames based on scroll %
4. **Interpolation:** Smooth transitions between frames using requestAnimationFrame
5. **Performance:** Each frame ~20-50KB WebP, total ~10-20MB for full experience

Code Pattern (Simplified):

```

const canvas = document.getElementById('scroll-canvas');
const ctx = canvas.getContext('2d');
const frames = []; // Array of Image objects
let currentFrame = 0;

// Load all frames
for (let i = 0; i < 400; i++) {
  const img = new Image();
  img.src = `frames/frame-${i.toString().padStart(4, '0')}.webp`;
  frames.push(img);
}

// Update frame on scroll
window.addEventListener('scroll', () => {
  const scrollPercent = window.scrollY / (document.body.scrollHeight - window.innerHeight);
  currentFrame = Math.floor(scrollPercent * (frames.length - 1));
  ctx.drawImage(frames[currentFrame], 0, 0, canvas.width, canvas.height);
});

```

Data Models (Inferred Schema)

Users Table

```

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(255),
  subscription_tier VARCHAR(50) DEFAULT 'free', -- free, pro, enterprise
  created_at TIMESTAMP DEFAULT NOW(),
  api_keys JSON, -- Encrypted storage for user's AI API keys
  monthly_credits INTEGER DEFAULT 2 -- Free tier: 2 sites/month
);

```

Projects Table

```

CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  title VARCHAR(255) NOT NULL,
  prompt TEXT, -- Original user prompt
  template_id UUID REFERENCES templates(id),
  conversation_history JSON, -- Chat messages for iterative editing
  generated_code TEXT, -- Final HTML/CSS/JS
  video_urls TEXT[], -- Array of generated video URLs
  frame_data JSON, -- Metadata about extracted frames
  status VARCHAR(50) DEFAULT 'draft', -- draft, processing, completed, published
  settings JSON, -- FPS, resolution, animation speed
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

```

Templates Table

```
CREATE TABLE templates (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(255) NOT NULL,  
  category VARCHAR(100), -- SaaS, E-commerce, Portfolio, etc.  
  preview_url TEXT,  
  base_code TEXT, -- Starting HTML/CSS/JS  
  system_prompt TEXT, -- AI instructions for this template style  
  metadata JSON, -- Colors, fonts, featured sections  
  is_public BOOLEAN DEFAULT true,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Video_Jobs Table

```
CREATE TABLE video_jobs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,  
  prompt TEXT NOT NULL,  
  provider VARCHAR(50), -- 'runway', 'kling', 'sora'  
  status VARCHAR(50) DEFAULT 'queued', -- queued, processing, completed, failed  
  video_url TEXT,  
  frame_count INTEGER,  
  processing_time_ms INTEGER,  
  error_message TEXT,  
  created_at TIMESTAMP DEFAULT NOW(),  
  completed_at TIMESTAMP  
);
```

Part 3: Product Requirements Document (PRD)

1. Vision & Goals

Product Vision

Build an AI-powered 3D scroll website builder that empowers non-technical users to create cinematic, production-ready websites through natural language prompts and visual editing, reducing time-to-launch from weeks to minutes.

Business Objectives

- 1. **Acquisition:** Reach 10,000 free users in first 6 months
- 2. **Conversion:** Achieve 5% free-to-paid conversion rate (\$49/month tier)
- 3. **Retention:** Maintain 80%+ monthly active user rate
- 4. **Expansion:** Launch marketplace for community templates by Month 12

Success Metrics

Metric	Target	Measurement
User Signups	10K in 6 months	Weekly cohort tracking

Metric	Target	Measurement
Time to First Site	< 5 minutes	Product analytics
Site Generation Success Rate	> 95%	Error logs + support tickets
Paid Conversion Rate	5% within 30 days	Subscription dashboard
Net Promoter Score (NPS)	> 50	Quarterly user surveys
Generated Site Quality	4.5/5 avg rating	Post-export user feedback

2. Target Users & Personas

Primary Persona: Sarah - Freelance Designer

- **Demographics:** 28 years old, freelance web designer, moderate coding skills
- **Pain Points:**
 - Clients expect impressive animations but she lacks advanced JS/WebGL skills
 - Framer is expensive (\$30/month per site), Webflow has steep learning curve
 - Manual 3D animation in After Effects → web is time-consuming
- **Goals:**
 - Deliver premium-looking websites quickly to impress clients
 - Spend more time on design strategy, less on technical implementation
 - Maintain competitive pricing without sacrificing quality
- **Use Case:** Uses Draftly presets as starting points, customizes with chat editor, exports for client hosting

Secondary Persona: Marcus - Startup Founder

- **Demographics:** 34 years old, technical background but focused on product, not marketing sites
- **Pain Points:**
 - Agency quotes for landing pages: \$5K-\$15K
 - DIY WordPress/Wix sites look generic and dated
 - No time to learn modern web animation frameworks
- **Goals:**
 - Launch impressive product landing page before investor pitch
 - Convey "premium tech startup" brand through visual design
 - Iterate quickly based on user feedback
- **Use Case:** Describes product vision in prompt, AI generates landing page, iterates with chat commands, publishes to Vercel

Tertiary Persona: Jessica - Marketing Manager

- **Demographics:** 31 years old, non-technical, manages 5-person marketing team
- **Pain Points:**
 - Dev team backlog is 2-3 months for simple landing page updates
 - Campaign-specific landing pages need fast turnaround
 - Budget constraints limit agency usage
- **Goals:**
 - Launch campaign landing pages independently without dev team
 - A/B test different visual approaches for conversion optimization
 - Maintain brand consistency across campaigns
- **Use Case:** Uses template gallery for quick starts, customizes copy/images, exports for hosting on company CDN

3. User Stories & Acceptance Criteria

Epic 1: AI Website Generation

US-1.1: As a user, I want to describe my desired website in natural language so that AI can generate a custom site matching my vision

- **Acceptance Criteria:**
 - Prompt input accepts 50-500 characters
 - AI generates initial preview within 60 seconds
 - Generated site includes hero, features, and CTA sections minimum
 - User can regenerate with modified prompt

US-1.2: As a user, I want to choose from preset templates so that I can start with a proven design

- **Acceptance Criteria:**
 - Gallery displays 7+ templates with live previews
 - Each template shows industry, style tags, and example use case
 - Clicking template opens builder with pre-loaded content
 - Templates are filterable by industry and style

US-1.3: As a user, I want to see a real-time preview of my scroll animation so that I can validate before export

- **Acceptance Criteria:**
 - Preview canvas updates within 3 seconds of changes
 - Scroll behavior matches exported code exactly
 - Preview supports mobile device emulation
 - Frame loading progress indicator visible

Epic 2: Iterative Chat Editing

US-2.1: As a user, I want to edit content through chat commands so that I can make changes without learning code

- **Acceptance Criteria:**
 - Natural language commands like "change hero text to..." work
 - AI maintains context across 10+ message conversation
 - Changes apply within 10 seconds
 - Undo/redo functionality available

US-2.2: As a user, I want to swap images/videos so that I can personalize the design

- **Acceptance Criteria:**
 - Drag-and-drop file upload (max 50MB)
 - Supports JPG, PNG, WebP, MP4 formats
 - AI auto-crops to fit design aspect ratio
 - CDN hosting URL provided for external use

Epic 3: 3D Scroll Animation Creation

US-3.1: As a user, I want to generate cinematic 3D scroll animations from prompts so that my site feels premium

- **Acceptance Criteria:**
 - Text prompt generates 8-second transition video
 - Supports image-to-video mode (user uploads start/end frames)
 - Video processing completes within 2-5 minutes
 - Extracted frames total < 20MB for performance

US-3.2: As a user, I want to chain multiple animations for longer scrolls so that I can create complex narratives

- **Acceptance Criteria:**
 - Support up to 5 video segments per scroll
 - Frame transitions blend smoothly at boundaries
 - Total animation length: 40 seconds maximum
 - Each segment independently editable

Epic 4: Export & Deployment

US-4.1: As a user, I want to download production-ready code so that I can host on my own infrastructure

- **Acceptance Criteria:**
 - ZIP contains index.html, styles.css, script.js, /frames directory
 - Code is minified and optimized

- Includes README with deployment instructions
- Total bundle size < 25MB

US-4.2: As a user, I want one-click deployment to Vercel/Netlify so that I can publish without technical setup

- **Acceptance Criteria:**
 - OAuth integration with Vercel/Netlify accounts
 - Custom domain mapping supported
 - SSL certificate auto-provisioned
 - Deploy completes within 60 seconds

4. Non-Functional Requirements

Performance

- Page load time: < 2 seconds on 3G connection
- Time to Interactive (TTI): < 3 seconds
- Scroll animation: 60 FPS on desktop, 30 FPS on mobile
- Frame preload: Complete within 5 seconds on broadband

Scalability

- Support 10,000 concurrent users in builder interface
- Handle 1,000 simultaneous video generation jobs
- CDN serves 1M+ frame requests per day with < 100ms latency
- Database queries: < 50ms for 95th percentile

Security

- All data encrypted at rest (AES-256) and in transit (TLS 1.3)
- User API keys stored in encrypted vault (AWS KMS/HashiCorp Vault)
- Rate limiting: 60 requests/minute per user
- GDPR/CCPA compliant data handling

Reliability

- System uptime: 99.9% (max 43 minutes downtime/month)
- Video generation success rate: > 95%
- Automated backups: Every 6 hours, retained 30 days
- Disaster recovery: RTO < 4 hours, RPO < 1 hour

Accessibility

- WCAG 2.1 AA compliance
- Keyboard navigation for all builder features

- Screen reader support with ARIA labels
- Color contrast ratios > 4.5:1

5. Out of Scope (V1)

- **Advanced 3D:** WebGL-based 3D objects, GLTF model imports
- **Animations:** Lottie, Rive, or custom animation timeline editors
- **CMS Integration:** WordPress, Contentful, or headless CMS connectors
- **E-commerce:** Shopping cart, payment processing, inventory management
- **Multi-page Sites:** Limited to single-page scrollers in V1
- **Team Collaboration:** Real-time co-editing, commenting, version control
- **Custom Code:** Direct HTML/CSS/JS editing within builder interface
- **Mobile App Builder:** iOS/Android native or React Native exports

6. Feature Prioritization

Feature	Priority	Rationale	Effort
AI Prompt → Website	P0 (MVP)	Core value proposition	High
Preset Templates	P0 (MVP)	Reduces time-to-first-site	Medium
Frame-based Scroll Animation	P0 (MVP)	Unique differentiator	High
Chat-based Editing	P0 (MVP)	Lowers barrier for non-technical users	Medium
ZIP Export	P0 (MVP)	Users need code ownership	Low
Video Generation (Runway/Kling)	P1 (Launch)	Premium feature, complex integration	High
Multi-video Chaining	P1 (Launch)	Extends use cases	Medium
One-click Deploy	P1 (Launch)	Removes hosting friction	Medium
FPS Adjustment	P2 (Post-launch)	Performance optimization for power users	Low
Template Marketplace	P3 (Future)	Community-driven growth	High
Custom Domains	P1 (Launch)	Professional use requirement	Low
Analytics Integration	P2 (Post-launch)	User insights for iteration	Medium

Part 4: Technical Requirements Document (TRD)

1. System Architecture

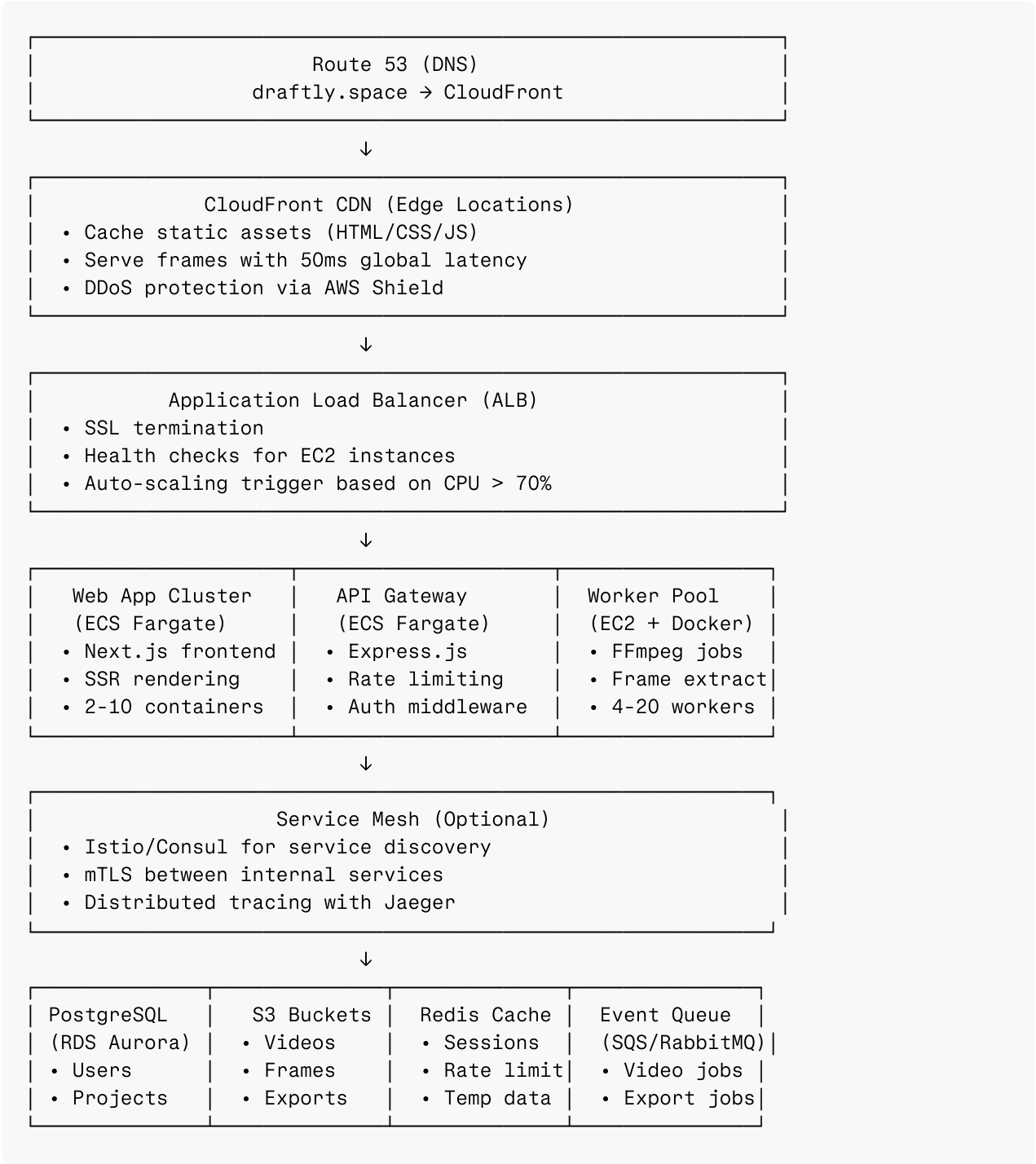
Architecture Pattern: Microservices with Event-Driven Communication

High-Level Components:

1. Web Application (Next.js 15 + React 19)

- 2. API Gateway (Node.js + Express)
- 3. AI Services (Claude Sonnet 4.0 + GPT-4)
- 4. Video Generation Service (Runway/Kling API wrapper)
- 5. Frame Processing Service (FFmpeg + Sharp)
- 6. Storage Layer (PostgreSQL + S3 + Redis)
- 7. CDN (CloudFront for global frame delivery)^{[1][12]}

Deployment Architecture:



2. Technology Stack Recommendations

Frontend

- **Framework:** Next.js 15 (App Router) + React 19[^13]
 - *Rationale:* SSR for SEO, streaming for real-time AI responses, Vercel deployment optimization
- **Styling:** Tailwind CSS v4
 - *Rationale:* Utility-first for rapid prototyping, excellent tree-shaking for bundle size
- **UI Components:** shadcn/ui + Radix UI
 - *Rationale:* Accessible, customizable, TypeScript-native
- **State Management:** Zustand + TanStack Query[^13]
 - *Rationale:* Zustand for global state, TanStack for server state & caching
- **Animation:** Framer Motion
 - *Rationale:* Declarative animations, scroll-linked animations, gesture support
- **Canvas Rendering:** Vanilla Canvas API (no Three.js for frame playback)[^5]
 - *Rationale:* Lower overhead, simpler implementation, better mobile performance

Backend

- **API Framework:** Express.js v5 on Node.js 20 LTS
 - *Rationale:* Mature ecosystem, excellent async support, streaming responses
- **Authentication:** NextAuth.js v5 (GitHub, Google OAuth)[^14]
 - *Rationale:* Built-in Next.js integration, session management, JWT support
- **Database ORM:** Prisma v6
 - *Rationale:* Type-safe queries, automatic migrations, excellent DX
- **File Upload:** Uploadthing or AWS SDK for S3
 - *Rationale:* Direct S3 uploads with pre-signed URLs, no server bottleneck
- **Task Queue:** BullMQ (Redis-backed)[^8]
 - *Rationale:* Reliable job processing, retries, priorities, rate limiting

AI & Video Services

- **LLM Provider:** Anthropic Claude Sonnet 4.0 (primary) + OpenAI GPT-4 (fallback)[^15][13]
 - *Rationale:* Claude excels at code generation, GPT-4 as backup reduces vendor lock-in
- **Video Generation:** Runway Gen-3 Turbo (primary) + Kling 3.0 Pro (alternative)[^16][9][^8]
 - *Rationale:* Runway has best quality/speed ratio, Kling as cost-effective alternative
 - *API Wrapper:* [AIMLAPI.com](https://aimlapi.com) for unified interface[^17][16]
- **Frame Processing:** FFmpeg (WASM for client-side + native for server-side)[^10][5]
 - *Rationale:* Industry standard, hardware acceleration support, extensive codec support

- **Image Optimization:** Sharp (Node.js) for WebP conversion[^10]
 - *Rationale:* 3x faster than ImageMagick, excellent quality at low file sizes

Data Layer

- **Primary Database:** PostgreSQL 16 (AWS RDS Aurora Serverless v2)
 - *Rationale:* ACID compliance, JSON support, automatic scaling, point-in-time recovery
- **Object Storage:** AWS S3 + CloudFront CDN[^12][18][^11]
 - *Rationale:* 99.999999999% durability, global CDN, versioning, lifecycle policies
- **Cache:** Redis 7 (ElastiCache)
 - *Rationale:* Sub-millisecond latency, pub/sub for real-time features, session storage
- **Search:** PostgreSQL Full-Text Search (upgrade to Elasticsearch if > 100K projects)
 - *Rationale:* Avoid additional infrastructure complexity in V1

DevOps & Monitoring

- **Hosting:** AWS (ECS Fargate for apps, EC2 for workers)
 - *Alternative:* Vercel for frontend + Railway for backend (faster MVP deployment)
- **CI/CD:** GitHub Actions + AWS CodeDeploy
 - *Rationale:* Free for public repos, native GitHub integration, Docker support
- **Monitoring:** Datadog or New Relic
 - *Rationale:* APM for backend, RUM for frontend, custom dashboards
- **Error Tracking:** Sentry
 - *Rationale:* Source maps, breadcrumbs, release tracking
- **Logging:** CloudWatch Logs + Logstash + Elasticsearch (ELK Stack)
 - *Rationale:* Centralized logging, powerful search, visualization with Kibana

3. API Design

RESTful API Endpoints

Authentication & Users

POST	/api/auth/signup	# Create new user account
POST	/api/auth/login	# Email/password login
POST	/api/auth/oauth/google	# OAuth Google redirect
GET	/api/users/me	# Get current user profile
PATCH	/api/users/me	# Update user profile
GET	/api/users/me/credits	# Check remaining monthly credits

Projects (CRUD)

POST	/api/projects	# Create new project from prompt
GET	/api/projects	# List user's projects (paginated)

```
GET    /api/projects/:id          # Get single project details
PATCH /api/projects/:id          # Update project metadata
DELETE /api/projects/:id          # Soft-delete project
POST   /api/projects/:id/duplicate # Clone existing project
```

AI Chat Editing

```
POST   /api/projects/:id/chat      # Send chat message, get AI response
GET     /api/projects/:id/chat      # Get conversation history
POST    /api/projects/:id/apply     # Apply AI-suggested code changes
POST    /api/projects/:id/undo      # Undo last change
```

Video Generation

```
POST   /api/videos/generate      # Create new video generation job
      Body: { prompt, model, duration, ratio, project_id }
GET     /api/videos/:job_id/status # Poll job status
GET     /api/videos/:job_id/result # Get completed video URL
POST    /api/videos/:job_id/extract # Trigger frame extraction
      Returns: { frame_urls[], total_frames, estimated_size_mb }
```

Frame Management

```
GET     /api/projects/:id/frames    # Get all frame URLs for project
POST    /api/projects/:id/frames/optimize # Re-process frames (resize, compress)
DELETE  /api/projects/:id/frames/:index # Delete specific frame
```

Templates

```
GET     /api/templates             # List all public templates
GET     /api/templates/:id          # Get template code + metadata
POST    /api/templates/:id/fork     # Create project from template
GET     /api/templates/categories   # Get template categories/tags
```

Export & Deploy

```
POST    /api/projects/:id/export    # Generate ZIP file
      Returns: { download_url, expires_at, file_size_mb }
POST    /api/projects/:id/deploy     # Deploy to Vercel/Netlify
      Body: { provider, custom_domain?, env_vars? }
GET     /api/projects/:id/deploy/status # Check deployment status
```

WebSocket Events (Real-time Features)

```
WS      /ws/projects/:id           # Real-time project updates
      Events:
      - code_update: { section, new_code }
      - video_progress: { job_id, percent_complete }
```

```
- frame_ready: { frame_index, url }
- chat_message: { role, content, timestamp }
```

API Response Format (Standardized)

```
{
  "success": true,
  "data": { /* response payload */ },
  "meta": {
    "timestamp": "2026-06-12T13:39:00Z",
    "request_id": "req_abc123",
    "rate_limit": {
      "remaining": 58,
      "reset_at": "2026-06-12T14:00:00Z"
    }
  },
  "error": null // Only present if success = false
}
```

4. Database Schema (Complete)

Entity-Relationship Diagram (ERD) - Key Relationships:

```
users (1) —< (many) projects
projects (1) —< (many) video_jobs
projects (1) —< (many) chat_messages
projects (many) >— (1) templates
users (1) —< (many) api_keys
```

Detailed Schema Definitions:

```
-- Users & Authentication
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT, -- NULL if OAuth-only
  name VARCHAR(255),
  avatar_url TEXT,
  subscription_tier VARCHAR(50) DEFAULT 'free' CHECK (subscription_tier IN ('free',
  subscription_expires_at TIMESTAMP,
  monthly_credits INTEGER DEFAULT 2,
  credits_reset_at TIMESTAMP DEFAULT (NOW() + INTERVAL '1 month'),
  oauth_provider VARCHAR(50), -- 'google', 'github', null
  oauth_id VARCHAR(255),
  email_verified BOOLEAN DEFAULT false,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  deleted_at TIMESTAMP -- Soft delete
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_oauth ON users(oauth_provider, oauth_id);
```



```

-- User API Keys (for AI providers)
CREATE TABLE api_keys (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  provider VARCHAR(50) NOT NULL, -- 'openai', 'anthropic', 'runway'
  encrypted_key TEXT NOT NULL, -- AES-256 encrypted
  key_name VARCHAR(100), -- User-defined label
  is_active BOOLEAN DEFAULT true,
  last_used_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_api_keys_user ON api_keys(user_id, provider);

-- Templates
CREATE TABLE templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE NOT NULL,
  description TEXT,
  category VARCHAR(100), -- 'saas', 'ecommerce', 'portfolio', 'ai', 'travel'
  tags TEXT[], -- Array of searchable tags
  preview_url TEXT, -- CloudFront URL to preview GIF/video
  thumbnail_url TEXT,
  base_html TEXT NOT NULL,
  base_css TEXT NOT NULL,
  base_js TEXT NOT NULL,
  system_prompt TEXT, -- AI instructions for maintaining template style
  metadata JSONB DEFAULT '{}', -- { colors, fonts, sections, ... }
  usage_count INTEGER DEFAULT 0,
  is_public BOOLEAN DEFAULT true,
  is_featured BOOLEAN DEFAULT false,
  created_by UUID REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_templates_category ON templates(category);
CREATE INDEX idx_templates_public ON templates(is_public) WHERE is_public = true;
CREATE INDEX idx_templates_featured ON templates(is_featured) WHERE is_featured = true;

-- Projects
CREATE TABLE projects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  template_id UUID REFERENCES templates(id) ON DELETE SET NULL,
  title VARCHAR(255) NOT NULL DEFAULT 'Untitled Project',
  slug VARCHAR(255), -- For custom URLs
  initial_prompt TEXT, -- User's first prompt
  current_html TEXT,
  current_css TEXT,
  current_js TEXT,
  conversation_history JSONB DEFAULT '[]', -- Array of chat messages
  video_urls TEXT[], -- Array of S3 URLs for generated videos
  frame_metadata JSONB DEFAULT '{}', -- { total_frames, fps, format, total_size_mb }

```

```

settings JSONB DEFAULT '{"fps": 30, "resolution": "1920x1080"}',
status VARCHAR(50) DEFAULT 'draft' CHECK (status IN ('draft', 'processing', 'complete')),
last_exported_at TIMESTAMP,
published_url TEXT, -- If deployed to Vercel/Netlify
view_count INTEGER DEFAULT 0,
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),
deleted_at TIMESTAMP
);

CREATE INDEX idx_projects_user ON projects(user_id);
CREATE INDEX idx_projects_status ON projects(status);
CREATE INDEX idx_projects_template ON projects(template_id);

-- Chat Messages (Conversation History)
CREATE TABLE chat_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
  role VARCHAR(20) NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
  content TEXT NOT NULL,
  code_changes JSONB, -- If message resulted in code updates
  attachments TEXT[], -- URLs to uploaded images/files
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_chat_messages_project ON chat_messages(project_id, created_at);

-- Video Generation Jobs
CREATE TABLE video_jobs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  prompt TEXT NOT NULL,
  provider VARCHAR(50) NOT NULL, -- 'runway', 'kling', 'sora'
  model VARCHAR(100), -- 'gen-3-turbo', 'kling-3.0-pro'
  input_type VARCHAR(50) DEFAULT 'text', -- 'text', 'image_to_video'
  input_image_url TEXT, -- If image-to-video mode
  duration_seconds INTEGER DEFAULT 8,
  aspect_ratio VARCHAR(20) DEFAULT '16:9',
  status VARCHAR(50) DEFAULT 'queued' CHECK (status IN ('queued', 'processing', 'complete')),
  external_job_id VARCHAR(255), -- Provider's job ID for status polling
  video_url TEXT, -- S3 URL of completed video
  frame_count INTEGER,
  processing_time_ms INTEGER,
  cost_usd DECIMAL(10, 4), -- Track API costs
  error_message TEXT,
  retry_count INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),
  started_at TIMESTAMP,
  completed_at TIMESTAMP
);

CREATE INDEX idx_video_jobs_project ON video_jobs(project_id);
CREATE INDEX idx_video_jobs_status ON video_jobs(status);
CREATE INDEX idx_video_jobs_user ON video_jobs(user_id);

```

```

-- Frames (Individual extracted frames from videos)
CREATE TABLE frames (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  video_job_id UUID REFERENCES video_jobs(id) ON DELETE CASCADE,
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
  frame_index INTEGER NOT NULL, -- 0-399 for 400-frame sequence
  s3_key TEXT NOT NULL, -- frames/{project_id}/{video_job_id}/frame-0001.webp
  cdn_url TEXT NOT NULL, -- CloudFront URL for fast delivery
  file_size_bytes INTEGER,
  width INTEGER,
  height INTEGER,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_frames_video_job ON frames(video_job_id, frame_index);
CREATE INDEX idx_frames_project ON frames(project_id);

-- Exports (Generated ZIP downloads)
CREATE TABLE exports (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  export_type VARCHAR(50) DEFAULT 'frontend', -- 'frontend', 'fullstack'
  s3_key TEXT NOT NULL,
  download_url TEXT NOT NULL, -- Pre-signed S3 URL (expires in 24h)
  file_size_mb DECIMAL(10, 2),
  expires_at TIMESTAMP DEFAULT (NOW() + INTERVAL '24 hours'),
  download_count INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_exports_project ON exports(project_id);
CREATE INDEX idx_exports_user ON exports(user_id);

-- Deployments (Vercel/Netlify integrations)
CREATE TABLE deployments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  provider VARCHAR(50) NOT NULL, -- 'vercel', 'netlify'
  deployment_url TEXT NOT NULL,
  custom_domain TEXT,
  status VARCHAR(50) DEFAULT 'deploying' CHECK (status IN ('deploying', 'ready', 'failed')),
  external_deployment_id VARCHAR(255), -- Provider's deployment ID
  logs TEXT,
  created_at TIMESTAMP DEFAULT NOW(),
  completed_at TIMESTAMP
);

CREATE INDEX idx_deployments_project ON deployments(project_id);

-- Usage Analytics (Track feature usage for product insights)
CREATE TABLE usage_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  event_type VARCHAR(100) NOT NULL, -- 'project_created', 'video_generated', 'export'

```

```

    project_id UUID,
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_usage_events_user ON usage_events(user_id, created_at);
CREATE INDEX idx_usage_events_type ON usage_events(event_type, created_at);

```

5. Security Considerations

Authentication & Authorization

- **JWT Strategy:** Short-lived access tokens (15 min) + long-lived refresh tokens (7 days)
- **OAuth Scopes:** Request minimal permissions (email, profile) from providers
- **API Key Storage:** Encrypt user's AI provider keys with AWS KMS^[19]
- **RBAC:** Roles = [free, pro, enterprise, admin] with feature flags per tier

Data Protection

- **PII Encryption:** Email, name, API keys encrypted at rest
- **SQL Injection Prevention:** Parameterized queries via Prisma ORM
- **XSS Protection:** CSP headers, input sanitization, React auto-escaping
- **CSRF Protection:** SameSite cookies + CSRF tokens for state-changing requests

API Security

- **Rate Limiting:** 60 req/min per user, 10 req/min per IP for unauthenticated^[20]
- **Input Validation:** Zod schemas on all API endpoints
- **File Upload:** Virus scanning (ClamAV), file type whitelisting, size limits (50MB)
- **Secrets Management:** AWS Secrets Manager or Doppler for env vars

Infrastructure Security

- **Network Isolation:** Private subnets for databases, public for load balancers only
- **Security Groups:** Strict ingress rules (443 only from internet, DB access from app tier only)
- **IAM Least Privilege:** Service accounts with minimal required permissions
- **Logging & Monitoring:** CloudTrail for AWS API calls, GuardDuty for threat detection

6. Performance Optimization

Frontend Optimizations

- **Code Splitting:** Dynamic imports for chat interface, template gallery
- **Image Optimization:** Next.js Image component with WebP, lazy loading
- **Frame Preloading:** Batch load frames (20 at a time) to avoid browser connection limits^[5]
- **Service Worker:** Cache frames in IndexedDB for offline playback

- **Bundle Size:** Target < 200KB initial JS, use tree-shaking aggressively

Backend Optimizations

- **Database Query Optimization:**
 - Indexes on all foreign keys and frequently queried columns
 - Connection pooling (min 10, max 100 connections)
 - Read replicas for analytics queries
- **Caching Strategy:**
 - Redis for session data (TTL: 7 days)
 - CDN caching for frames (TTL: 1 year, cache invalidation via purge API)
 - Application-level caching for templates (refresh every 5 minutes)
- **Async Processing:** All long-running tasks (video gen, frame extraction) in background queues^[8]

CDN & Asset Delivery^{[11][12]}

- **Global Distribution:** CloudFront edge locations in 50+ countries
- **Compression:** Brotli for text files, WebP for images
- **HTTP/2 Push:** Push critical CSS/JS to reduce round trips
- **Adaptive Bitrate:** Serve different frame qualities based on connection speed (future)

7. Scalability Plan

Phase 1: MVP (0-1K users)

- **Architecture:** Monolithic Next.js app on Vercel, PostgreSQL on Supabase free tier
- **Estimated Cost:** \$50-100/month (AI API usage + storage)
- **Bottlenecks:** Single database instance, limited background workers

Phase 2: Growth (1K-10K users)

- **Architecture:** Separate frontend (Vercel) + backend (AWS ECS), managed PostgreSQL (RDS)
- **Scaling:**
 - Horizontal: 2-5 ECS Fargate containers behind ALB
 - Database: Connection pooling via PgBouncer, read replicas for analytics
 - Workers: 4-10 EC2 instances for video processing
- **Estimated Cost:** \$500-1500/month
- **Bottlenecks:** Video generation API rate limits, frame storage costs

Phase 3: Scale (10K-100K users)

- **Architecture:** Microservices (Chat, Video, Export as separate services), Aurora Serverless
- **Scaling:**

- Horizontal: Auto-scaling groups (2-20 containers per service)
- Database: Aurora auto-scaling, DynamoDB for session data
- Workers: Kubernetes cluster (EKS) with 10-50 pods
- Cache: Redis Cluster (3-node replication)
- **Estimated Cost:** \$5K-15K/month
- **Optimizations:** CDN 95%+ cache hit rate, batch video processing

Phase 4: Enterprise (100K+ users)

- **Architecture:** Multi-region deployment, sharded databases
- **Scaling:**
 - Horizontal: 50+ containers, traffic split across US-East, EU-West, Asia-Pacific
 - Database: Sharding by user_id, CockroachDB for global distribution
 - Workers: Serverless functions (Lambda) for burst capacity
- **Estimated Cost:** \$25K-50K/month
- **Advanced Features:** Real-time collaboration via WebSockets, custom model fine-tuning

8. Deployment Strategy

Development Environment

```
# docker-compose.yml for local development
services:
  app:
    build: .
    ports: ["3000:3000"]
    environment:
      DATABASE_URL: postgresql://localhost:5432/draftly_dev
      REDIS_URL: redis://localhost:6379
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: draftly_dev
  redis:
    image: redis:7-alpine
  worker:
    build: ./worker
    depends_on: [postgres, redis]
```

CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/deploy.yml
name: Deploy to Production
on:
  push:
    branches: [main]
jobs:
```

```

test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm install && npm test
    - run: npm run lint
build:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - run: docker build -t draftly:${{ github.sha }} .
    - run: docker push ecr.amazonaws.com/draftly:${{ github.sha }}
deploy:
  needs: build
  runs-on: ubuntu-latest
  steps:
    - run: aws ecs update-service --cluster prod --service draftly-api --force-new

```

Zero-Downtime Deployment

1. Build new Docker image with updated code
2. Push to ECR (Elastic Container Registry)
3. ECS rolls out new containers (5 at a time, wait 60s health check)
4. Old containers drain connections for 120s before termination
5. Rollback strategy: Keep last 3 image versions, instant revert if error rate > 1%

Monitoring Dashboards

- **System Health:** CPU, memory, disk usage per container
- **Application Metrics:** Request latency (p50, p95, p99), error rates, throughput
- **Business Metrics:** New signups, projects created, exports downloaded, revenue
- **AI Usage:** Video generation success rate, average processing time, cost per generation

Alerting Rules

- **Critical:** Error rate > 5% for 5 minutes → PagerDuty
- **Warning:** P95 latency > 2s for 10 minutes → Slack
- **Info:** Deployment completed → Slack
- **Budget:** Monthly AWS spend > \$10K → Email finance team

Part 5: Gap Analysis - Signhify vs. Draftly

Current State Analysis: Signhify

Strengths:[^2]

- 1. **Modern Tech Stack:** TanStack Start, React 19, TypeScript, Tailwind v4 - production-ready foundation
- 2. **3D Capabilities:** Three.js + React Three Fiber integrated - can render 3D scenes
- 3. **Design System:** Radix UI components, Framer Motion animations - polished UI
- 4. **Backend Ready:** Supabase + PostgreSQL configured - data persistence in place
- 5. **Portfolio Showcase:** Demonstrates technical credibility with real projects

Identified Gaps:

Feature	Draftly	Signhify	Gap Severity	Effort to Close
AI Prompt-to-Website	✔ Core feature	✗ Missing	Critical	6-8 weeks
Frame-based Scroll Animation	✔ 400 frames/video	✗ Uses Three.js	Critical	4-6 weeks
Video Generation Integration	✔ Runway/Kling	✗ Not implemented	Critical	3-4 weeks
Chat-based Editor	✔ Iterative editing	✗ Missing	High	4-5 weeks
Template Gallery	✔ 7+ presets	✗ No templates	High	2-3 weeks
Code Export	✔ ZIP download	✗ Not applicable	High	1-2 weeks
FFmpeg Frame Extraction	✔ Server-side	✗ Missing	Medium	2 weeks
One-click Deploy	✔ Vercel/Netlify	✗ Manual only	Medium	1-2 weeks
Subscription System	✔ Free/Pro tiers	✗ No monetization	Low	2-3 weeks

Technical Divergence Points

1. 3D Rendering Approach

- **Draftly:** Frame sequence (400 WebP images) → Canvas rendering → No runtime 3D[^3][5]
 - **Pros:** Universal compatibility, predictable performance, smaller bundle
 - **Cons:** Less interactive, larger asset size, no real-time modifications
- **Signhify:** Three.js WebGL → Real-time 3D rendering → GPU-accelerated
 - **Pros:** Interactive elements, particles, lighting effects, smaller initial load
 - **Cons:** Performance varies by device, compatibility issues (older browsers), larger JS bundle

Recommendation: Implement both approaches:

- Use frame-based for hero sections (cinematic wow factor)
- Use Three.js for interactive product showcases (where user control matters)
- Offer users choice in builder: "Cinematic scroll" vs "Interactive 3D"

2. Architecture Pattern

- **Draftly:** No-code SaaS platform (builder generates sites for others)
- **Signhify:** Studio portfolio + services site (showcases your capabilities)

Pivot Strategy:

- **Option A:** Keep Signhify as portfolio, build "Signhify AI" as separate product (URL: ai.signhify.com)
- **Option B:** Rebrand Signhify as full builder platform, migrate portfolio to subdomain (portfolio.signhify.com)
- **Recommended:** Option A - clear separation of concerns, easier to iterate on AI product

3. AI Integration Maturity

- **Draftly:** Core product built around AI code generation^{[1][6]}
- **Signhify:** AI used for development (Lovable platform) but not exposed to end-users

Implementation Roadmap:

1. Week 1-2: Integrate Anthropic Claude API for code generation^{[15][13]}
2. Week 3-4: Build chat interface component with streaming responses^[^13]
3. Week 5-6: Develop prompt engineering system for consistent HTML/CSS output
4. Week 7-8: Implement code diff application (apply AI suggestions to existing code)

Architectural Recommendations for Signhify

Immediate Changes (Month 1):

```
Current: Monolithic Lovable App
      ↓
Target: Signhify Studio (Portfolio)  +  Signhify AI (Builder SaaS)
      |— Static Next.js site           |— Chat-based editor
      |— Showcase past projects        |— AI code generation
      |— Contact/booking forms         |— Template gallery
                                      |— Export/deploy features
```

Database Schema Additions:

```
-- Add to existing Signhify database
CREATE TABLE ai_projects (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id),
  prompt TEXT,
  generated_html TEXT,
  generated_css TEXT,
  generated_js TEXT,
  conversation JSONB,
  status VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW())
```

```
);

CREATE TABLE video_generations (
  id UUID PRIMARY KEY,
  project_id UUID REFERENCES ai_projects(id),
  prompt TEXT,
  provider VARCHAR(50), -- 'runway', 'klimg'
  video_url TEXT,
  frame_count INTEGER,
  status VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW()
);
```

Tech Stack Convergence:

Keep from Signhify:

- ✓ TanStack Start/Router (faster than Next.js App Router)
- ✓ Supabase (easier auth + real-time vs. custom Express)
- ✓ Tailwind v4 (newest features)
- ✓ Framer Motion (animation library)

Add from Draftly Analysis:

- + FFmpeg (via ffmpeg.wasm for client-side)
- + Anthropic SDK (for chat-based editing)
- + Runway/Kling API wrappers
- + Canvas-based frame renderer
- + BullMQ (for background video jobs)

Migration Path: Signhify → Signhify AI

Phase 1: Foundation (Weeks 1-4)

- [] Set up separate subdomain (ai.signhify.app)
- [] Create new Supabase tables for AI projects
- [] Integrate Claude API for basic code generation
- [] Build MVP chat interface (text input → code output)
- [] Test with 10 beta users from your network

Phase 2: Video Integration (Weeks 5-8)

- [] Set up Runway/Kling API accounts
- [] Build video generation job queue (BullMQ)
- [] Implement ffmpeg frame extraction (server-side EC2)
- [] Create Canvas scroll animation component
- [] Upload frames to S3 + CloudFront

Phase 3: Templates & Export (Weeks 9-12)

- [] Design 3 starter templates (SaaS, Portfolio, Landing)

- [] Build template gallery UI
- [] Implement ZIP export with optimized assets
- [] Add one-click Vercel deployment
- [] Launch to Product Hunt / Hacker News

Phase 4: Monetization (Weeks 13-16)

- [] Implement Stripe subscription (Free: 2/month, Pro: Unlimited @ \$49/mo)
- [] Add usage analytics dashboard
- [] Create referral program (give 1 free credit for each referral)
- [] Build in-app upgrade prompts

Resource Requirements

Team Composition (Recommended):

- **1 Full-Stack Engineer (You):** 60% backend (APIs, video processing), 40% frontend (chat UI)
- **1 AI/ML Engineer (Contract):** Prompt engineering, model fine-tuning, quality control
- **1 Designer (Part-time):** Template design, UI polish, marketing assets
- **Total Cost:** \$15K-20K/month for 4-6 months to MVP

Infrastructure Budget:

Month 1-2 (Development):

- Vercel Pro: \$20/mo
- Supabase Pro: \$25/mo
- OpenAI/Claude: \$200/mo (testing)
- Runway/Kling: \$500/mo (testing)
- Total: ~\$750/mo

Month 3-6 (Launch):

- Vercel Enterprise: \$200/mo
- Supabase Pro: \$25/mo
- AI APIs: \$2K/mo (500 sites @ \$4/site)
- AWS S3+CloudFront: \$500/mo
- Monitoring (Datadog): \$100/mo
- Total: ~\$3K/mo

Post-Launch (1K users):

- Scale linearly: \$5-7K/mo
- Break-even: ~150 paid users (\$49/mo)

Part 6: Implementation Roadmap - MVP in 16 Weeks

Phase 0: Pre-Development (Week 0)

Objective: Validate assumptions, set up infrastructure

Tasks:

- [] User research: Interview 10 potential customers (freelancers, founders, marketers)
- [] Competitive analysis: Deep dive on Framer, Webflow, v0.dev pricing/features
- [] Technical proof-of-concept: Test Runway API → FFmpeg → Canvas rendering end-to-end
- [] Domain purchase + brand assets (logo, color palette, typography)
- [] AWS/Vercel account setup, infrastructure-as-code (Terraform/CDK)

Deliverables:

- User persona docs
- Technical feasibility report
- Brand style guide v1.0

Phase 1: Core Infrastructure (Weeks 1-4)

Objective: Build foundational architecture and AI code generation

Week 1-2: Backend Foundation

- [] Set up Next.js 15 project with TypeScript + Tailwind^[13]
- [] Configure Supabase: Auth (email + Google OAuth), PostgreSQL tables^[21]
- [] Create Express API endpoints: /api/projects, /api/chat
- [] Integrate Claude Sonnet 4.0 with streaming responses^{[14][15]}
- [] Build prompt template system (few-shot examples for HTML/CSS generation)

Week 3-4: Chat Interface

- [] Design chat UI (shadcn/ui components: Input, ScrollArea, Card)
- [] Implement real-time streaming (Server-Sent Events or WebSockets)
- [] Add conversation history storage in PostgreSQL
- [] Build code preview pane with syntax highlighting (Monaco Editor or Shiki)
- [] Test chat flow: Prompt → AI response → Apply code → Preview updates

Acceptance Criteria:

- User can describe a website in natural language
- AI generates valid HTML/CSS/JS within 30 seconds
- Code previews in real-time iframe sandbox

- Chat maintains context across 5+ message exchanges

Phase 2: Video & Animation Pipeline (Weeks 5-8)

Objective: Integrate AI video generation and frame-based scroll animation

Week 5-6: Video Generation

- [] Set up Runway Gen-3 Turbo API integration^{[16][8]}
- [] Build job queue system (BullMQ + Redis)^[^8]
- [] Create `/api/videos/generate` endpoint with status polling
- [] Implement video webhook handler for completion notifications
- [] Store video URLs in S3 with CloudFront distribution^{[12][11]}

Week 7-8: Frame Extraction & Rendering

- [] Deploy EC2 instance with FFmpeg for server-side processing^{[10][5]}
- [] Build frame extraction service: MP4 → 400 WebP images
- [] Upload frames to S3 with batch operations (parallel uploads)
- [] Create Canvas scroll animation component (React)^[^5]
- [] Implement frame preloading strategy (batches of 20)
- [] Test on mobile devices for performance (target 30fps)

Acceptance Criteria:

- User provides text prompt → Runway generates 8s video in < 3 minutes
- FFmpeg extracts 400 frames in < 60 seconds
- Canvas scroll animation plays at 60fps on desktop, 30fps on mobile
- Total frame bundle size < 20MB (WebP compression)

Phase 3: Templates & Export (Weeks 9-12)

Objective: Build template gallery and code export functionality

Week 9-10: Template System

- [] Design 3 starter templates: SaaS (dark), E-commerce (light), Portfolio (colorful)
- [] Create template metadata schema: name, category, tags, preview_url
- [] Build template gallery UI with filters and search
- [] Implement "Fork Template" → New Project workflow
- [] Add system prompts for each template style (maintain design consistency)

Week 11-12: Export & Deploy

- [] Build ZIP generation service: Bundle HTML/CSS/JS/frames into archive

- ☐ Implement S3 pre-signed URLs for secure downloads (24h expiry)
- ☐ Create deployment integrations: Vercel (OAuth) + Netlify (API tokens)
- ☐ Add custom domain configuration UI
- ☐ Build export history page (list past downloads)

Acceptance Criteria:

- Users can browse 3 templates with live previews
- "Fork Template" creates editable project in < 10 seconds
- ZIP export completes in < 30 seconds, total size < 30MB
- One-click Vercel deploy works end-to-end (OAuth → Git push → Live URL)

Phase 4: Polish & Launch (Weeks 13-16)

Objective: Add monetization, optimize performance, launch publicly

Week 13-14: Monetization

- ☐ Integrate Stripe Checkout + Customer Portal^[^20]
- ☐ Implement credit system: Free (2/month), Pro (\$49/mo unlimited)
- ☐ Add usage dashboard: Credits remaining, projects count, bandwidth used
- ☐ Build upgrade prompts: "Out of credits" modal with Stripe link
- ☐ Set up webhook handlers for subscription events (cancel, renew, upgrade)

Week 15: Performance Optimization

- ☐ Lighthouse audit: Target scores (Performance: 90+, Accessibility: 95+, SEO: 100)
- ☐ Optimize frame loading: Implement IntersectionObserver for lazy loading
- ☐ Add Redis caching for template data (5-minute TTL)
- ☐ Compress API responses with Brotli
- ☐ Set up CloudFront cache rules (max-age: 1 year for frames)

Week 16: Launch Preparation

- ☐ Write launch blog post + demo video (60-90 seconds)
- ☐ Create Product Hunt teaser page + Discord community
- ☐ Set up analytics: Mixpanel for events, Google Analytics for traffic
- ☐ Prepare support infrastructure: Intercom chat, help docs site
- ☐ Launch sequence: Personal network (Day 1) → Product Hunt (Day 3) → Hacker News (Day 7)

Launch Day Checklist:

- ☐ Monitoring dashboards live (Datadog + Sentry)
- ☐ Load testing completed (1000 concurrent users)

- ☐ Error handling graceful (user-friendly messages)
- ☐ Support team ready (24h response time SLA)
- ☐ Backup/disaster recovery tested
- ☐ Legal: Privacy Policy, Terms of Service, Cookie Notice

Post-Launch Roadmap (Weeks 17-24)

Month 5 (Weeks 17-20): Iterate Based on Feedback

- ☐ Analyze usage data: Which templates most popular? Where do users drop off?
- ☐ Add 2-3 new templates based on user requests
- ☐ Improve AI prompt understanding (fine-tune with real user prompts)
- ☐ Build feedback widget: "Was this AI response helpful?"
- ☐ Optimize for SEO: Landing pages for each template category

Month 6 (Weeks 21-24): Advanced Features

- ☐ Multi-video chaining (40s+ scroll experiences)
- ☐ Custom font uploads (Google Fonts integration)
- ☐ Image-to-video mode (user uploads keyframes)
- ☐ Collaborative editing (share project with team members)
- ☐ API access for developers (embed builder in their apps)

Year 1 Milestones:

- Month 7-9: Template marketplace (creators sell templates, 70/30 revenue split)
- Month 10-12: White-label offering for agencies (\$499/mo, custom branding)
- Month 12+: Enterprise features (SSO, custom models, on-premise deployment)

Risk Mitigation

Top Risks & Mitigation Strategies:

Risk	Likelihood	Impact	Mitigation
AI API costs exceed budget	High	High	Implement strict rate limiting, cache common responses, negotiate volume discounts
Video generation too slow	Medium	High	Use multiple providers (Runway + Kling), show progress indicators, set expectations ("2-3 minutes")
Frame storage costs balloon	Medium	Medium	Aggressive compression (WebP quality: 75), delete frames after 30 days inactivity, tiered storage (S3 → Glacier)
Competitors copy features	High	Medium	Focus on UX polish and brand, file provisional patents on frame-based rendering technique
Technical co-founder leaves	Low	High	Document everything, modular architecture, hire contractor backup

Risk	Likelihood	Impact	Mitigation
Runway/Kling API changes	Medium	High	Abstract provider interface, maintain fallback to older models, diversify across 3+ providers

Success Metrics (First 6 Months)

Acquisition:

- Target: 10,000 signups
- Channels: Product Hunt (2K), Organic search (3K), Social media (2K), Referrals (3K)
- CAC Target: < \$5 (mostly organic)

Activation:

- Target: 70% complete first project within 7 days
- Onboarding: Tutorial video, example prompts, template recommendations

Retention:

- Target: 40% return to create 2nd project within 30 days
- Tactics: Email drip campaign, showcase community creations, new template alerts

Revenue:

- Target: \$25K MRR by Month 6 (500 paid users @ \$49/mo)
- Conversion: 5% free → paid within 30 days
- LTV:CAC Ratio: 10:1 (LTV \$600, CAC \$60)

Referral:

- Target: 30% viral coefficient (each user brings 0.3 new users)
- Mechanism: "Built with Draftly" badge on exported sites, referral credits program

Part 7: Cinematic 3D Immersive Website/UI Design Prompt

Master Prompt for AI-Generated Cinematic 3D Website Design

Context: You are designing a cutting-edge, cinematic 3D immersive website for [PROJECT_NAME], a [PRODUCT/SERVICE DESCRIPTION]. The target audience is [DEMOGRAPHIC: e.g., premium tech buyers, creative professionals, B2B decision-makers] who expect a visually stunning, narrative-driven experience that communicates value through motion, depth, and interaction.

Objectives:

1. **Showcase Value:** Primary CTA is [ACTION: e.g., book demo, start free trial, download whitepaper]

2. **Tell a Story:** Guide users through a 3-5 scene narrative journey
3. **Premium Feel:** Convey quality through cinematic lighting, smooth animations, tactile interactions
4. **High Performance:** 60fps on desktop, 30fps on mobile, < 3s initial load

Tech Constraints:

- Frontend: React 19 + Three.js/React Three Fiber (for WebGL 3D) OR Canvas API (for frame-based)
- Rendering: Desktop & mobile, progressive enhancement for WebGL fallback
- Performance Budget: 500KB initial JS, 2MB total assets (lazy load rest)
- Accessibility: WCAG 2.1 AA, keyboard navigation, screen reader labels

Scene-by-Scene Breakdown

Scene 1: Hero / Entry Experience (0-20% scroll)

Visual Brief:

- **Environment:** Deep space with floating geometric particles (icosahedrons, torus knots)
- **Camera Motion:** Slow dolly forward (5 seconds), subtle parallax on scroll
- **Lighting:**
 - Key light: Soft white (color temp 5500K) from top-right, simulates sun
 - Rim light: Cool blue (#4A90E2) from back-left for depth separation
 - Fill light: Warm amber (#FFA500) from front-bottom, 30% intensity
- **Textures:**
 - Particles: Metallic material with roughness 0.3, environment map reflection
 - Background: Gradient from deep navy (#0A1929) to dark purple (#1E0A3C)
- **Typography:**
 - Headline: 120px bold sans-serif (Inter/Geist), white with subtle text shadow
 - Word-by-word reveal on scroll (stagger: 100ms per word)
 - Subheadline: 24px light, 60% opacity, fade in after headline completes

Interaction Design:

- **Mouse Parallax:** Particles move inverse to cursor (dampening: 0.1, max offset: 50px)
- **Scroll Behavior:** Headline fades out at 15% scroll, particles disperse outward
- **CTA Button:**
 - Glass morphism style: backdrop-blur(10px), border 1px solid rgba(255,255,255,0.2)
 - Hover: Glow effect (box-shadow: 0 0 30px rgba(74,144,226,0.6))

- Click: Particle burst animation (20 particles radiating from center)

State Diagram:

```
[Idle] → (Mouse Move) → [Parallax Shift] → (Mouse Stop) → [Ease Back to Center]
[Idle] → (Scroll Down) → [Fade Out + Particle Disperse] → Scene 2 Transition
```

Code Snippet (React Three Fiber):

```
import { Canvas, useFrame } from '@react-three/fiber'
import { OrbitControls, Environment } from '@react-three/drei'
import { useScroll } from 'framer-motion'

function Scene1() {
  const { scrollYProgress } = useScroll()
  const meshRef = useRef()

  useFrame(({ mouse, clock }) => {
    // Parallax on mouse move
    meshRef.current.rotation.x = mouse.y * 0.2
    meshRef.current.rotation.y = mouse.x * 0.2

    // Float animation
    meshRef.current.position.y = Math.sin(clock.elapsedTime) * 0.5

    // Fade out on scroll
    meshRef.current.material.opacity = 1 - scrollYProgress.get()
  })

  return (
    <mesh ref={meshRef}>
      <icosahedronGeometry args=[[1, 2]] />
      <meshStandardMaterial
        color="#4A90E2"
        metalness={0.8}
        roughness={0.3}
        transparent
      />
    </mesh>
  )
}
```

Scene 2: Feature Showcase (20-50% scroll)

Visual Brief:

- **Environment:** Minimalist white space, subtle grid floor (perspective distortion)
- **3D Object:** Product mockup (laptop, phone, or abstract representation)
 - Model: Low-poly style, ~50K triangles maximum
 - Material: Matte with edge highlights (Fresnel shader)
- **Camera Motion:** Arc rotation around product (120° over scroll duration)

- **Lighting:**
 - Studio setup: 3-point lighting (key, fill, rim)
 - HDRI environment map for realistic reflections
- **Feature Callouts:**
 - 3 floating cards appear sequentially (stagger: 200ms)
 - Each card: Icon + Title + 2-line description
 - Card style: Neumorphism (soft shadows, subtle depth)

Interaction Design:

- **Scroll-Linked Camera:** Camera position tied to scroll % ($\text{scrollYProgress} * 120^\circ$ rotation)
- **Hover on Cards:** Card lifts 10px, shadow intensifies, arrow icon animates right
- **Click on Card:** Expand to full-screen modal with detailed feature breakdown

Asset Requirements:

- Product 3D model: GLTF/GLB format, < 2MB, Draco compressed
- Feature icons: SVG, 24x24px, monochrome for easy color swapping
- Background texture: Seamless grid PNG, 1024x1024px, subtle opacity (10%)

Performance Notes:

- Use LOD (Level of Detail): Show low-poly model on mobile, high-poly on desktop
- Lazy load 3D model: Show placeholder box until GLTF loads
- Reduce shadow quality on mobile (shadowMapSize: 512 vs 2048 on desktop)

Scene 3: Data Visualization / Impact (50-75% scroll)

Visual Brief:

- **Environment:** Dark mode, starfield background (particle system, 1000+ stars)
- **Data Viz:**
 - Animated bar chart / line graph rising from bottom
 - Numbers count up (Intersection Observer trigger when in viewport)
 - 3D columns with glass material (refraction, chromatic aberration)
- **Camera Motion:** Slow push-in toward data (creates focus/importance)
- **Typography:**
 - Large stat numbers: 96px, gradient fill (linear-gradient(135deg, #667eea 0%, #764ba2 100%))
 - Label text: 18px uppercase, tracking: 0.1em, color: rgba(255,255,255,0.7)

Interaction Design:

- **Scroll Trigger:** Chart animates in when 30% visible (Intersection Observer)

- **Hover on Data Point:** Tooltip appears with exact value, data point scales 1.2x
- **Mobile:** Replace 3D chart with 2D Canvas version (better performance)

Shader Effect (Custom WebGL):

```
// Fragment shader for glass data columns
varying vec3 vNormal;
varying vec3 vPosition;

void main() {
    vec3 viewDir = normalize(cameraPosition - vPosition);
    float fresnel = pow(1.0 - dot(viewDir, vNormal), 3.0);

    vec3 baseColor = vec3(0.3, 0.5, 0.9); // Light blue
    vec3 glowColor = vec3(0.6, 0.8, 1.0); // Bright cyan

    vec3 finalColor = mix(baseColor, glowColor, fresnel);
    gl_FragColor = vec4(finalColor, 0.8); // 80% opacity for glass effect
}
```

Scene 4: Social Proof / Testimonials (75-90% scroll)

Visual Brief:

- **Environment:** Soft gradient background (light to dark, top to bottom)
- **3D Element:** Floating testimonial cards in 3D space (carousel layout)
- **Camera Motion:** Lateral tracking shot (follows cards as they slide)
- **Card Design:**
 - Frosted glass material: backdrop-filter: blur(20px)
 - Avatar image: Circular, 80px diameter, subtle glow
 - Quote text: 18px serif font (Merriweather/Lora), italic
 - Company logo: Grayscale, 24px height

Interaction Design:

- **Auto-Rotate:** Cards shift every 5 seconds (infinite loop)
- **Drag to Navigate:** User can swipe/drag cards horizontally
- **Hover on Card:** Card scales 1.05x, z-position forward (depth separation)
- **Accessibility:** Keyboard arrows (←→) navigate cards, focus outline visible

Animation Sequence:

1. Cards enter from right (translate3d(100%, 0, 0))
2. Settle into position with spring physics (react-spring: mass 1, tension 170, friction 26)
3. Exit to left with same easing
4. Next card follows immediately (no gap)

Scene 5: CTA / Final Scene (90-100% scroll)

Visual Brief:

- **Environment:** Return to minimal, focus on action
- **3D Element:** Pulsating orb or abstract shape (symbolizes "start")
- **Camera Motion:** Slow zoom into orb (creates tunnel effect)
- **Lighting:**
 - Orb emits light (emissive material, bloom post-processing)
 - Background darkens to pure black (vignette effect)
- **Typography:**
 - CTA headline: 72px, bold, centered, white
 - Button: 56px height, 240px width, rounded corners (border-radius: 28px)
 - Hover state: Background shifts from gradient to solid, shadow grows

Interaction Design:

- **Button Hover:**
 - Orb pulses faster (from 2s to 1s period)
 - Mouse cursor becomes custom graphic (arrow with sparkles)
 - Haptic feedback on mobile (navigator.vibrate(10))
- **Button Click:**
 - Orb explodes into particles (200 fragments)
 - White flash transition (duration: 300ms)
 - Navigate to form/dashboard

Post-Processing Effects:

- **Bloom:** Threshold 0.8, intensity 1.5, radius 0.5 (only on orb)
- **Film Grain:** Subtle noise overlay (opacity: 0.05) for texture
- **Chromatic Aberration:** On edges only (offset: 2px red, -2px cyan)

UI Component Library Specification

Navigation Bar

- **Style:** Transparent background, blurs content behind (backdrop-filter: blur(10px))
- **Layout:** Logo (left) | Nav Links (center) | CTA Button (right)
- **Scroll Behavior:** Shrink height from 80px → 60px at 100px scroll
- **States:**
 - Default: White text, no background

- Scrolled: Dark background (#0A1929, 90% opacity), white text
- Mobile: Hamburger menu (animated to X on click)

Buttons

- **Primary:** Gradient background (linear-gradient(135deg, #667eea 0%, #764ba2 100%)), white text, 16px padding, border-radius 8px
- **Secondary:** Transparent background, 1px white border, white text
- **Hover:** Scale 1.05, shadow appears (0 8px 24px rgba(102, 126, 234, 0.4))
- **Active:** Scale 0.98, shadow disappears (pressed effect)
- **Loading:** Spinner animation, text changes to "Loading..." with ellipsis pulsing

Cards

- **Container:** backdrop-filter: blur(20px), background: rgba(255,255,255,0.05), border: 1px solid rgba(255,255,255,0.1)
- **Shadow:** 0 8px 32px rgba(0,0,0,0.2)
- **Hover:** Transform translateY(-8px), shadow intensifies
- **Content Padding:** 32px all sides

Typography Scale

H1: 96px / 104px line-height, font-weight: 700 (Hero headlines)
H2: 60px / 68px, font-weight: 700 (Section titles)
H3: 36px / 44px, font-weight: 600 (Subsection titles)
Body: 18px / 28px, font-weight: 400 (Paragraph text)
Caption: 14px / 20px, font-weight: 400 (Labels, meta info)

Color Palette

```
/* Primary Gradient */
--gradient-primary: linear-gradient(135deg, #667eea 0%, #764ba2 100%);

/* Background */
--bg-dark: #0A1929;
--bg-mid: #1E0A3C;
--bg-light: #2D1B4E;

/* Text */
--text-primary: #FFFFFF;
--text-secondary: rgba(255,255,255,0.7);
--text-tertiary: rgba(255,255,255,0.5);

/* Accent */
--accent-blue: #4A90E2;
--accent-purple: #764ba2;
--accent-pink: #FF6B9D;

/* Semantic */
--success: #4ADE80;
```

```
--warning: #FCD34D;  
--error: #EF4444;
```

Asset Generation Guide

3D Models

- **Format:** GLTF/GLB (binary preferred for size)
- **Polycount:**
 - Hero objects: 50K-100K triangles (desktop), 10K-20K (mobile)
 - Background elements: < 5K triangles
- **Textures:**
 - Base Color: 2048x2048 PNG/JPG
 - Normal Map: 2048x2048 PNG
 - Roughness/Metallic: 1024x1024 PNG (combined in channels)
- **Compression:** Draco compression (reduces size by 70-90%)
- **Tools:** Blender (free), Cinema 4D, Maya

Video Backgrounds (if frame-based approach)

- **Resolution:** 1920x1080 (1080p), 3840x2160 (4K for hero only)
- **Frame Rate:** 30fps (mobile), 60fps (desktop)
- **Duration:** 8-12 seconds per scene
- **Codec:** H.264, CRF 18 (high quality), -preset slow
- **Format:** MP4 (maximum compatibility)

Images

- **Format:** WebP (primary), JPEG (fallback), PNG (if alpha needed)
- **Responsive Sizes:** 640w, 1024w, 1920w, 3840w (srcset)
- **Compression:** WebP quality 80, JPEG quality 85
- **Lazy Loading:** All images below fold (loading="lazy")

Icons & Graphics

- **Format:** SVG (inline for critical, external file for non-critical)
- **Viewbox:** 0 0 24 24 (standard icon size)
- **Stroke Width:** 1.5px-2px for consistency
- **Fill:**currentColor (inherits text color for easy theming)

Starter Project Skeleton

```
cinematic-3d-website/
├── public/
│   ├── models/
│   │   ├── hero-object.glb          # Hero scene 3D model
│   │   └── product-mockup.glb      # Feature showcase model
│   ├── videos/
│   │   └── hero-bg.mp4             # Optional: video background
│   ├── images/
│   │   ├── og-image.jpg            # Social sharing preview
│   │   └── favicon.ico
│   └── fonts/
│       ├── Inter-Variable.woff2    # Variable font for performance
│       └── Merriweather-Italic.woff2
├── src/
│   ├── components/
│   │   ├── scenes/
│   │   │   ├── Scene1Hero.tsx      # Hero with particles
│   │   │   ├── Scene2Features.tsx  # Product showcase
│   │   │   ├── Scene3Data.tsx      # Data visualization
│   │   │   ├── Scene4Social.tsx    # Testimonials carousel
│   │   │   └── Scene5CTA.tsx       # Final call-to-action
│   │   ├── ui/
│   │   │   ├── Button.tsx
│   │   │   ├── Card.tsx
│   │   │   ├── Navigation.tsx
│   │   │   └── ScrollProgress.tsx  # Progress bar indicator
│   │   ├── Canvas3D.tsx            # Three.js canvas wrapper
│   │   └── Layout.tsx
│   ├── shaders/
│   │   ├── glass.frag              # Fragment shader for glass material
│   │   ├── glass.vert              # Vertex shader
│   │   └── particle.frag           # Particle system shader
│   ├── hooks/
│   │   ├── useScroll.ts            # Custom scroll progress hook
│   │   ├── useMouseParallax.ts     # Mouse tracking for parallax
│   │   └── use3DModel.ts           # GLTF loader hook
│   ├── styles/
│   │   ├── globals.css             # Tailwind base + custom CSS
│   │   └── animations.css          # Keyframe animations
│   ├── lib/
│   │   ├── three-utils.ts          # Helper functions for Three.js
│   │   └── constants.ts            # Colors, breakpoints, timings
│   ├── pages/
│   │   └── index.tsx               # Main landing page
│   └── app/
│       ├── layout.tsx              # Root layout with font loading
│       └── page.tsx                # Home page (uses scenes)
├── tests/
│   ├── accessibility.test.ts       # WCAG compliance tests
│   ├── performance.test.ts         # Lighthouse score targets
│   └── visual-regression.test.ts    # Percy/Chromatic integration
├── .env.example                    # Environment variables template
├── next.config.js                  # Next.js configuration
└── tailwind.config.ts              # Tailwind + custom theme
```



```
├─ tsconfig.json          # TypeScript config
└─ package.json
```

Key Files Content:

src/components/Canvas3D.tsx:

```
'use client'
import { Canvas } from '@react-three/fiber'
import { Suspense } from 'react'
import { OrbitControls, Environment, Html } from '@react-three/drei'

export function Canvas3D({ children }: { children: React.ReactNode }) {
  return (
    <Canvas
      camera={{ position: [0, 0, 5], fov: 75 }}
      gl={{
        antialias: true,
        alpha: true,
        powerPreference: 'high-performance'
      }}
      dpr={[1, 2]} // Limit pixel ratio for performance
    >
      <Suspense fallback={
        <Html center>
          <div className="animate-spin text-white">Loading...</div>
        </Html>
      }>
        <Environment preset="city" /> {/* HDR lighting */}
        {children}
      </Suspense>
    </Canvas>
  )
}
```

src/hooks/useScroll.ts:

```
import { useScroll as useFramerScroll } from 'framer-motion'
import { useEffect, useState } from 'react'

export function useScroll() {
  const { scrollYProgress } = useFramerScroll()
  const [progress, setProgress] = useState(0)

  useEffect(() => {
    return scrollYProgress.onChange((latest) => {
      setProgress(latest)
    })
  }, [scrollYProgress])

  return {
    progress, // 0 to 1
    percentage: progress * 100, // 0 to 100
    isInView: (start: number, end: number) =>
```

```

    progress >= start && progress <= end
  }
}

```

tailwind.config.ts:

```

import type { Config } from 'tailwindcss'

const config: Config = {
  content: ['./src/**/*.{ts,tsx}'],
  theme: {
    extend: {
      colors: {
        primary: '#667eea',
        secondary: '#764ba2',
        dark: '#0A1929'
      },
      animation: {
        'float': 'float 3s ease-in-out infinite',
        'glow': 'glow 2s ease-in-out infinite alternate'
      },
      keyframes: {
        float: {
          '0%, 100%': { transform: 'translateY(0px)' },
          '50%': { transform: 'translateY(-20px)' }
        },
        glow: {
          '0%': { boxShadow: '0 0 20px rgba(102, 126, 234, 0.5)' },
          '100%': { boxShadow: '0 0 40px rgba(118, 75, 162, 0.8)' }
        }
      }
    }
  },
  plugins: []
}
export default config

```

Testing & Optimization Checklist

Visual Quality

- ☐ Lighting setup uses 3-point technique (key, fill, rim)
- ☐ Shadows enabled with appropriate resolution (2048 desktop, 1024 mobile)
- ☐ Anti-aliasing enabled (MSAA or FXAA)
- ☐ Color grading applied (subtle desaturation, contrast boost)
- ☐ Motion blur on fast camera movements (optional, performance cost)

Performance

- ☐ Lighthouse Scores: Performance > 90, Accessibility > 95, SEO > 95

- ☐ **Frame Rate:** 60fps sustained on desktop (M1 MacBook, RTX 3060), 30fps on mobile (iPhone 13)
- ☐ **Load Time:** FCP < 1.5s, LCP < 2.5s, CLS < 0.1
- ☐ **Bundle Size:** JS < 500KB initial, total < 2MB after lazy loading
- ☐ **Memory Usage:** < 200MB heap size, no memory leaks (test with DevTools)

Accessibility

- ☐ Keyboard navigation works for all interactive elements (Tab, Enter, Space)
- ☐ Focus indicators visible (outline: 2px solid #4A90E2, offset: 2px)
- ☐ ARIA labels on 3D scenes ("Interactive 3D visualization of product features")
- ☐ Color contrast ratios $\geq 4.5:1$ for text, $\geq 3:1$ for UI elements
- ☐ Screen reader announces scroll progress ("Section 2 of 5: Feature Showcase")
- ☐ Reduced motion support (prefers-reduced-motion: reduce) disables parallax, slows animations
- ☐ Alternative text descriptions for visual-only content

Cross-Browser Testing

- ☐ Chrome/Edge (latest 2 versions)
- ☐ Firefox (latest 2 versions)
- ☐ Safari (latest 2 versions, iOS + macOS)
- ☐ Samsung Internet (Android)
- ☐ WebGL fallback for unsupported browsers (show 2D version or static images)

Device Testing

- ☐ Desktop: 1920x1080, 2560x1440, 3840x2160
- ☐ Tablet: iPad (1024x768), iPad Pro (1366x1024)
- ☐ Mobile: iPhone 13 (390x844), Samsung S21 (360x800)
- ☐ Orientation: Portrait and landscape modes work correctly

SEO Optimization

- ☐ Open Graph meta tags (og:title, og:description, og:image, og:url)
- ☐ Twitter Card meta tags (twitter:card, twitter:title, twitter:image)
- ☐ JSON-LD structured data for organization/product
- ☐ Canonical URL set correctly
- ☐ Sitemap.xml includes all navigable pages
- ☐ Robots.txt allows indexing
- ☐ Page title < 60 characters, meta description < 160 characters

Final Deliverables Summary

Documentation:

1. **Scene Storyboard PDF:** Visual mockups of each scene with annotations
2. **Asset Manifest CSV:** List of all 3D models, videos, images with file sizes and sources
3. **Component Library Figma:** UI components with states and responsive breakpoints
4. **Animation Timing Sheet:** Keyframe timings, easing functions, trigger points

Code:

1. **GitHub Repository:** Complete Next.js project with all components
2. **Deployment Guide Markdown:** Step-by-step instructions for Vercel/Netlify
3. **Environment Variables Template:** `.env.example` with required API keys
4. **CI/CD Pipeline YAML:** GitHub Actions for automated testing + deployment

Assets:

1. **3D Models Package:** GLB files with textures, properly compressed
2. **Video Files:** MP4s for hero backgrounds, optimized for web
3. **Image Assets:** WebP + JPEG fallbacks in multiple sizes
4. **Font Files:** WOFF2 format, variable fonts preferred

This comprehensive prompt provides everything needed to build a cinematic, production-ready 3D website. Adapt the scene descriptions to your specific project's narrative, and use the technical specifications as a blueprint for implementation.

References

1. [3D Website Builder - Draftly](#) - Create stunning 3D immersive websites in minutes with AI, no coding required. Simply enter a prompt ...
2. [I Built a \\$10,000 Website Using AI](#) - I built a premium Apple-style scroll animation website using AI tools — here's the full step-by-step...
3. [ai-video-scroll-animation](#) | Skills M...
4. [How to Create Stunning 3D Scroll Animations for Premium Websites ...](#) - This article explains how to build high-quality 3D scrolling animations for websites using AI tools ...
5. [FFmpeg -> Image Sequence \(WebP\) -> Canvas + Scroll Logic](#) - 2,196 likes, 43 comments - artemiy.miller on March 9, 2026: "A guide to building Apple-style scroll-..."
6. [Draftly 3D Website Builder | No-Code Immersive Website Creation](#) - Build cinematic 3D websites from a single prompt — about 10× faster. AI motion, frame extraction, an...
7. [Comment "ANIMATE" and I'll send you the full guide with the exact ...](#) - 944 likes, 1,486 comments - gennaroautomates on March 16, 2026: "Comment "ANIMATE" and I'll send you..."

8. [Runway API v1 | Experimental API for AI services - UseAPI.net](#) - Experimental API for popular AI services by [useapi.net](#)
9. [Mar 5 '26 • Runway Videos | Experimental API for AI services - Experimental API for popular AI services by useapi.net](#)
10. [MP4 to WebP on macOS: One-Click ffmpeg Script Guide - Ima Studio](#) - Use a one-click macOS shell script with ffmpeg to convert MP4 to looping WebP animations, with auto-...
11. [Tutorial: Hosting on-demand streaming video with Amazon S3, Amazon CloudFront, and Amazon Route 53](#) - Walk through an example of how to configure an S3 bucket to host on-demand video streaming using Clo...
12. [How to Stream Video from AWS \(S3, Cloudfront, NextJS & Typescript\)](#) - Learn how to start a video streaming website/web application using Amazon S3, Amazon Cloudfront, Nex...
13. [Build Your AI Chat Platform | ChatHub | Next.js, OpenAI, Gemini, Claude, Ollama, Langchain | Part 1](#) - *[Build Your OWN AI Chat Platform | ChatHub! | Next.js, OpenAI, Gemini, Claude, LangChain, TanSta...](#)
14. [AI Chatbot Integration Guide: Build Intelligent Conversational ...](#) - A comprehensive guide to integrating AI chatbots into your applications using OpenAI, Anthropic Clau...
15. [GitHub - Lywald/MixAI: Multi-Model AI Chat](#) - Multi-Model AI Chat. Contribute to Lywald/MixAI development by creating an account on GitHub.
16. [Runway Gen-3 Turbo vs Kling Pro - AIMLAPI.com](#) - AI-powered video generation tools like Runway and Kling have emerged as leading solutions for conten...
17. [AI Video Generator API – Sora 2 | Veo 3 | Kling AI | Runway](#) - [All-in-One AI Video Generator API – Sora 2 + Veo 3 + Kling AI + Runway + Hailuo in one API key. Ch...](#)
18. [Set up S3 and stream video with AWS Cloudfront](#) - Socials
LinkedIn: <https://www.linkedin.com/in/thabish/>
Twitter: <https://twitter.com/DeveloperTak>
G...
19. [Building Chatbots with ChatGPT and Anthropic APIs 101](#) - This is a concise primer on constructing chatbots using large language models (LLM) like ChatGPT and...
20. [What is a Product Requirements Document \(PRD\)? - Agile - Atlassian](#) - A product requirements document (PRD) details the product's behavior, purpose, and features to align...
21. [Supabase : Building a Backend for AI Websites Without Coding](#) - In this episode of Everything Product, Sid and Funny explore Supabase, a powerful backend solution t...